Undoing Commits & Changes

In this section, we will discuss the available 'undo' Git strategies and commands. It is first important to note that Git does not have a traditional 'undo' system like those found in a word processing application. It will be beneficial to refrain from mapping Git operations to any traditional 'undo' mental model. Additionally, Git has its own nomenclature for 'undo' operations that it is best to leverage in a discussion. This nomenclature includes terms like reset, revert, checkout, clean, and more.

A fun metaphor is to think of Git as a timeline management utility. Commits are snapshots of a point in time or points of interest along the timeline of a project's history. Additionally, multiple timelines can be managed through the use of branches. ***When 'undoing' in Git, you are usually moving back in time, or to another timeline where mistakes didn't happen.***

This tutorial provides all of the necessary skills to work with previous revisions of a software project. First, it shows you how to explore old commits, then it explains the difference between reverting public commits in the project history vs. resetting unpublished changes on your local machine.

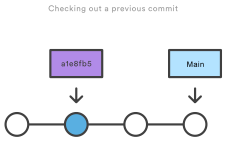
Finding what is lost: Reviewing old commits

The whole idea behind any version control system is to store “safe” copies of a project so that you never have to worry about irreparably breaking your code base. Once you’ve built up a project history of commits, you can review and revisit any commit in the history. One of the best utilities for reviewing the history of a Git repository is the `git log` command. In the example below, we use git log to get a list of the latest commits to a popular open-source graphics library.

```
git log --oneline
e2f9a78fe Replaced FlyControls with OrbitControls
d35ce0178 Editor: Shortcuts panel Safari support.
9dbe8d0cf Editor: Sidebar.Controls to Sidebar.Settings.Shortcuts. Clean up.
05c5288fc Merge pull request #12612 from TyLindberg/editor-controls-panel
0d8b6e74b Merge pull request #12805 from harto/patch-1
23b20c22e Merge pull request #12801 from gam0022/improve-raymarching-example-v2
fe78029f1 Fix typo in documentation
7ce43c448 Merge pull request #12794 from WestLangley/dev-x
17452bb93 Merge pull request #12778 from OndrejSpanel/unitTestFixes
b5c1b5c70 Merge pull request #12799 from dhritzkiv/patch-21
1b48ff4d2 Updated builds.
88adbcdf6 WebVRManager: Clean up.
2720fbb08 Merge pull request #12803 from dmarcos/parentPoseObject
9ed629301 Check parent of poseObject instead of camera
219f3eb13 Update GLTFLoader.js
15f13bb3c Update GLTFLoader.js
6d9c22a3b Update uniforms only when onWindowResize
881b25b58 Update ProjectionMatrix on change aspect
```

Each commit has a unique SHA-1 identifying hash. These IDs are used to travel through the committed timeline and revisit commits. By default, `git log` will only show commits for the currently selected branch. It is entirely possible that the commit you're looking for is on another branch. You can view all commits across all branches by executing `git log --branches=*`. The command git branch is used to view and visit other branches. Invoking the command, `git branch -a` will return a list of all known branch names. One of these branch names can then be logged using `git log <branch_name>`.

When you have found a commit reference to the point in history you want to visit, you can utilize the `git checkout` command to visit that commit. `Git checkout` is an easy way to "load" any of these saved snapshots onto your development machine. During the normal course of development, the `HEAD` usually points to `main` or some other local branch, but ***when you check out a previous commit,*** *`HEAD`* ***no longer points to a branch–it points directly to a commit. This is called a "detached*** *`HEAD`* ***" state,*** and it can be visualized as the following:



Checking out an old file does not move the `HEAD` pointer. It remains on the same branch and same commit, avoiding a 'detached head' state. You can then commit the old version of the file in a new snapshot as you would any other changes. So, in effect, this usage of `git checkout` on a file, serves as a way to revert back to an old version of an individual file. For more information on these two modes visit the git checkout page

Viewing an old revision

This example assumes that you've started developing a crazy experiment, but you're not sure if you want to keep it or not. To help you decide, you want to take a look at the state of the project before you started your experiment. First, you'll need to find the ID of the revision you want to see.

```
git log --oneline
```

Let's say your project history looks something like the following:

```
b7119f2 Continue doing crazy things
872fa7e Try something crazy
a1e8fb5 Make some important changes to hello.txt
435b61d Create hello.txt
9773e52 Initial import
```

You can use `git checkout` to view the "Make some import changes to hello.txt" commit as follows:

```
git checkout a1e8fb5
```

This makes your working directory match the exact state of the `a1e8fb5` commit. You can look at files, compile the project, run tests, and even edit files without worrying about losing the current state of the project. Nothing you do in here will be saved in your repository. To continue developing, you need to get back to the “current” state of your project:

```
git checkout main
```

This assumes that you're developing on the default `main` branch. Once you’re back in the `main` branch, you can use either git revert or git reset to undo any undesired changes.

Undoing a committed snapshot

There are technically several different strategies to 'undo' a commit. The following examples will assume we have a commit history that looks like:

```
git log --oneline
872fa7e Try something crazy
a1e8fb5 Make some important changes to hello.txt
435b61d Create hello.txt
9773e52 Initial import
```

We will focus on undoing the `872fa7e Try something crazy` commit. Maybe things got a little too crazy.

How to undo a commit with git checkout

Using the `git checkout` command we can checkout the previous commit, `a1e8fb5`, putting the repository in a state before the crazy commit happened. Checking out a specific commit will put the repo in a "detached `HEAD`" state. This means you are no longer working on any branch. In a detached state, any new commits you make will be orphaned when you change branches back to an established branch. Orphaned commits are up for deletion by Git's garbage collector. The garbage collector runs on a configured interval and permanently destroys orphaned commits. To prevent orphaned commits from being garbage collected, we need to ensure we are on a branch.

From the detached `HEAD` state, we can execute `git checkout -b new_branch_without_crazy_commit`. This will create a new branch named `new_branch_without_crazy_commit` and switch to that state. The repo is now on a new history timeline in which the `872fa7e` commit no longer exists. At this point, we can continue work on this new branch in which the `872fa7e` commit no longer exists and consider it 'undone'. Unfortunately, if you need the previous branch, maybe it was your `main` branch, this undo strategy is not appropriate. Let's look at some other 'undo' strategies. For more information and examples review our in-depth git checkout discussion.

How to undo a public commit with git revert

Let's assume we are back to our original commit history example. The history that includes the `872fa7e` commit. This time let's try a revert 'undo'. If we execute `git revert HEAD`, Git will create a new commit with the inverse of the last commit. This adds a new commit to the current branch history and now makes it look like:

```
git log --oneline
e2f9a78 Revert "Try something crazy"
872fa7e Try something crazy
a1e8fb5 Make some important changes to hello.txt
435b61d Create hello.txt
9773e52 Initial import
```

At this point, we have again technically 'undone' the `872fa7e` commit. Although `872fa7e` still exists in the history, the new `e2f9a78` commit is an inverse of the changes in `872fa7e`. Unlike our previous checkout strategy, we can continue using the same branch. This solution is a satisfactory undo. This is the ideal 'undo' method for working with public shared repositories. If you have requirements of keeping a curated and minimal Git history this strategy may not be satisfactory.

How to undo a commit with git reset

For this undo strategy we will continue with our working example. git reset is an extensive command with multiple uses and functions. If we invoke `git reset --hard a1e8fb5` the commit history is reset to that specified commit. Examining the commit history with `git log` will now look like:

```
git log --oneline
a1e8fb5 Make some important changes to hello.txt
435b61d Create hello.txt
9773e52 Initial import
```

The log output shows the `e2f9a78` and `872fa7e` commits no longer exist in the commit history. At this point, we can continue working and creating new commits as if the 'crazy' commits never happened. This method of undoing changes has the cleanest effect on history. Doing a reset is great for local changes however it adds complications when working with a shared remote repository. If we have a shared remote repository that has the `872fa7e` commit pushed to it, and we try to `git push` a branch where we have reset the history, Git will catch this and throw an error. Git will assume that the branch being pushed is not up to date because of it's missing commits. In these scenarios, `git revert` should be the preferred undo method.

Undoing the last commit

In the previous section, we discussed different strategies for undoing commits. These strategies are all applicable to the most recent commit as well. In some cases though, you might not need to remove or reset the last commit. Maybe it was just made prematurely. In this case you can amend the most recent commit. Once you have made more changes in the working directory and staged them for commit by using git add, you can execute `git commit --amend`. This will have Git open the configured system editor and let you modify the last commit message. The new changes will be added to the amended commit.

Undoing uncommitted changes

Before changes are committed to the repository history, they live in the staging index and the working directory. You may need to undo changes within these two areas. The staging index and working directory are internal Git state management mechanisms. For more detailed information on how these two mechanisms operate, visit the git reset page which explores them in depth.

The working directory

The working directory is generally in sync with the local file system. To undo changes in the working directory you can edit files like you normally would using your favorite editor. Git has a couple utilities that help manage the working directory. There is the git clean command which is a convenience utility for undoing changes to the working directory. Additionally, `git reset` can be invoked with the `--mixed` or `--hard` options and will apply a reset to the working directory.

The staging index

The git add command is used to add changes to the staging index. `Git reset` is primarily used to undo the staging index changes. A `--mixed` reset will move any pending changes from the staging index back into the working directory.

Undoing public changes

When working on a team with remote repositories, extra consideration needs to be made when undoing changes. `Git reset` should generally be considered a 'local' undo method. A reset should be used when undoing changes to a private branch. This safely isolates the removal of commits from other branches that may be in use by other developers. Problems arise when a reset is executed on a shared branch and that branch is then pushed remotely with `git push`. Git will block the push in this scenario complaining that the branch being pushed is out of date from the remote branch as it is missing commits.

The preferred method of undoing shared history is `git revert`. A revert is safer than a reset because it will not remove any commits from a shared history. A revert will retain the commits you want to undo and create a new commit that inverts the undesired commit. This method is safer for

shared remote collaboration because a remote developer can then pull the branch and receive the new revert commit which undoes the undesired commit.

Summary

We covered many high-level strategies for undoing things in Git. It's important to remember that there is more than one way to 'undo' in a Git project. Most of the discussion on this page touched on deeper topics that are more thoroughly explained on pages specific to the relevant Git commands. The most commonly used 'undo' tools are git checkout, git revert, and git reset. Some key points to remember are:

- Once changes have been committed they are generally permanent
- Use `git checkout` to move around and review the commit history
- `git revert` is the best tool for undoing shared public changes
- `git reset` is best used for undoing local private changes

In addition to the primary undo commands, we took a look at other Git utilities: git log for finding lost commits git clean for undoing uncommitted changes git add for modifying the staging index.

Each of these commands has its own in-depth documentation. To learn more about a specific command mentioned here, visit the corresponding links.